Student Task Manager Architecture

Full-Stack Project Architecture Document

Task 1 — Project Architecture

1. Product Idea

Student Task Manager helps students organize assignments, coursework, exams, and personal study tasks. Users can add tasks, set due dates and priorities, mark tasks complete, and filter the list.

2. Target Users

School and university students who need a simple study planner.

3. Technology Stack

Frontend: React + Vite

Backend: Node.js + Express

Database: SQLite (planned)

Communication: REST API using JSON

4. Architecture Overview

Browser (React UI) → Express REST API → data access layer → SQLite database. The frontend renders screens and sends HTTP requests. The backend validates requests and applies business rules. SQLite stores task data persistently.

5. Data Model — Task

id: integer, primary key, auto-generated

title: string, required, 1–120 characters

description: optional string

dueDate: optional date

priority: low, medium, or high (default medium)

status: pending or completed (default pending)

createdAt: datetime

updatedAt: datetime

6. API Routes

GET /api/health — check API status

GET /api/tasks — list tasks; optional status filter

GET /api/tasks/:id — get one task

POST /api/tasks — create task

PUT /api/tasks/:id — update task

PATCH /api/tasks/:id/status — update status

DELETE /api/tasks/:id — delete task

7. Frontend Screens

Dashboard / Task List: shows tasks and filters.

Add Task: title, description, due date, priority.

Edit Task: update an existing task.

Task Details (optional): show all task information.

Empty / Error States: explain missing tasks or API connection errors.

8. Suggested Folder Structure

student-task-manager/

ARCHITECTURE.md

README.md

backend/src/server.js

backend/package.json

frontend/index.html

frontend/src/main.jsx

frontend/src/App.jsx

frontend/package.json

9. Validation and Security

Validate required fields and allowed priority/status values on the server. Use parameterized SQL when database integration is implemented. Keep secrets in environment variables and never commit .env files. Return clear HTTP error codes and JSON messages.

10. Development Plan

1. Set up frontend and backend.
2. Implement CRUD API routes.
3. Connect SQLite and create the tasks table.
4. Connect frontend forms and task list to the API.
5. Add filtering and validation.
6. Test workflows and add screenshots to the README.

11. Current Starter Status

The starter includes a working Express health endpoint and a React shell that checks API connectivity. Task CRUD routes and persistent database integration are defined as the next development steps.